Домашнє завдання 4

Apache Spark. Оптимізація та SparkUI

Три програми виконано локально у VS Code через Ubuntu WSL із використанням Spark 3.5.5. У SparkUI отримано відповідно 5, 8 і 7 завершених завдань. Нижче наведено скриншоти та пояснення результатів.

Частина 1 Один виклик collect

Запуск: ~/spark/bin/spark-submit part1.py

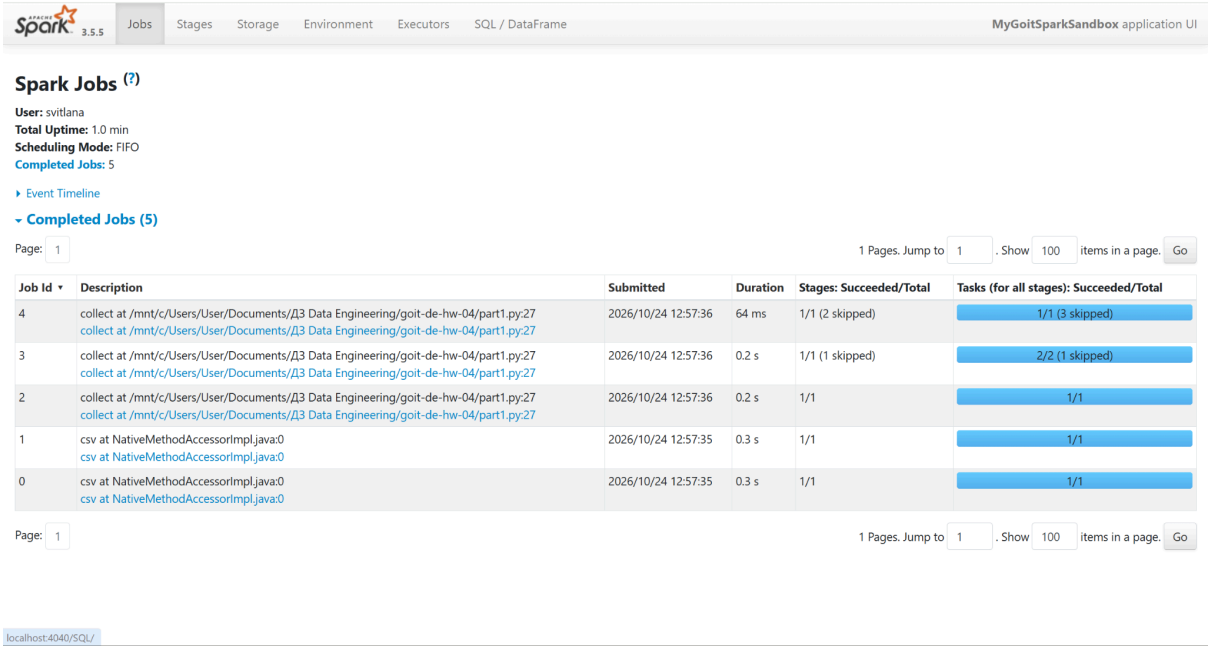


Рисунок 1. Частина 1 - Completed Jobs (завершені завдання): 5, номери від 0 до 4.

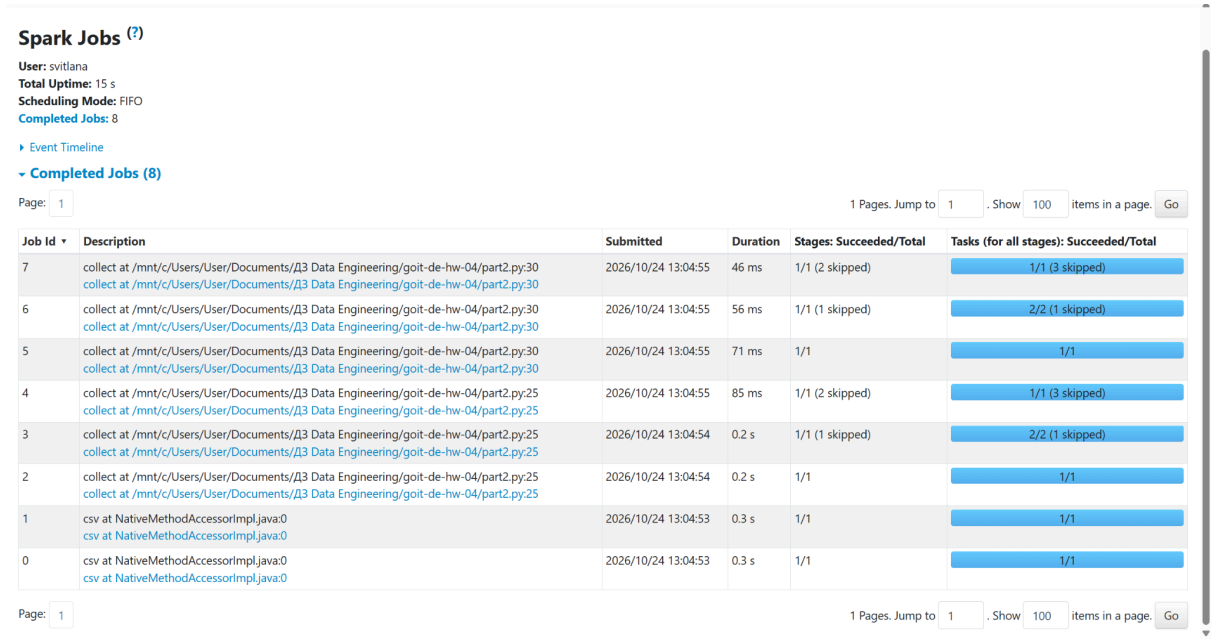
Пояснення результату

У цьому запуску Jobs (завдання) 0 і 1 пов’язані з читанням CSV та автоматичним визначенням схеми даних за допомогою inferSchema. Завдання 2-4 пов’язані з єдиним викликом collect() після всіх перетворень.

Перетворення виконуються ліниво: Spark спочатку формує план. Виклик collect() запускає обчислення, зокрема перерозподіл на дві частини через repartition(2), фільтрацію final_priority < 3, групування за unit_id, підрахунок і фільтрацію count > 2. Один collect() у цьому плані породжує декілька завдань.

Частина 2 Два виклики collect

Запуск: ~/spark/bin/spark-submit part2.py



Spark Jobs (?)

User: svitlana
Total Uptime: 15 s
Scheduling Mode: FIFO
Completed Jobs: 8

Event Timeline

Completed Jobs (8)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
7	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:30 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:30	2026/10/24 13:04:55	46 ms	1/1 (2 skipped)	1/1 (3 skipped)
6	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:30 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:30	2026/10/24 13:04:55	56 ms	1/1 (1 skipped)	2/2 (1 skipped)
5	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:30 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:30	2026/10/24 13:04:55	71 ms	1/1	1/1
4	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:25 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:25	2026/10/24 13:04:55	85 ms	1/1 (2 skipped)	1/1 (3 skipped)
3	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:25 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:25	2026/10/24 13:04:54	0.2 s	1/1 (1 skipped)	2/2 (1 skipped)
2	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:25 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part2.py:25	2026/10/24 13:04:54	0.2 s	1/1	1/1
1	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/10/24 13:04:53	0.3 s	1/1	1/1
0	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/10/24 13:04:53	0.3 s	1/1	1/1

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Рисунок 2. Частина 2 - Completed Jobs (завершені завдання): 8, номери від 0 до 7.

Пояснення результату

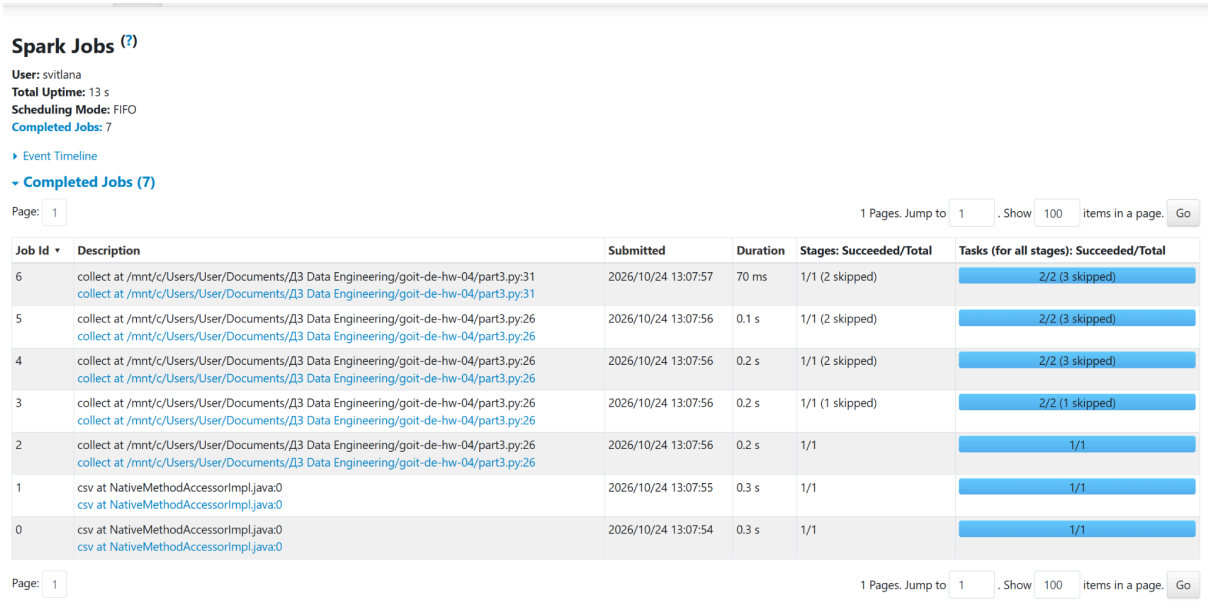
До програми додано проміжний collect() після groupBy().count(), перед фільтрацією count > 2. У SparkUI завдання 0 і 1 відповідають читанню CSV, завдання 2-4 першому collect() у рядку 25, а завдання 5-7 другому collect() у рядку 30.

Перший collect() обчислює проміжний результат, але не кешує його для подальшого використання цим DataFrame (таблицею даних Spark). Наступний collect() після додавання фільтра запускає окреме виконання плану. Тому в цьому запуску отримано на три завдання більше, ніж у частині 1: 2 + 3 + 3 = 8.

Отже, додаткова дія може збільшити обсяг роботи. Без кешування не слід очікувати, що проміжний результат попереднього collect() автоматично використовуватиметься наступною дією.

Частина 3 Використання cache

Запуск: ~/spark/bin/spark-submit part3.py



Spark Jobs (?)

User: svitlana
Total Uptime: 13 s
Scheduling Mode: FIFO
Completed Jobs: 7

▸ Event Timeline

▾ Completed Jobs (7)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
6	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:31 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:31	2026/10/24 13:07:57	70 ms	1/1 (2 skipped)	2/2 (3 skipped)
5	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26	2026/10/24 13:07:56	0.1 s	1/1 (2 skipped)	2/2 (3 skipped)
4	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26	2026/10/24 13:07:56	0.2 s	1/1 (2 skipped)	2/2 (3 skipped)
3	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26	2026/10/24 13:07:56	0.2 s	1/1 (1 skipped)	2/2 (1 skipped)
2	collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26 collect at /mnt/c/Users/User/Documents/ДЗ Data Engineering/goit-de-hw-04/part3.py:26	2026/10/24 13:07:56	0.2 s	1/1	1/1
1	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/10/24 13:07:55	0.3 s	1/1	1/1
0	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/10/24 13:07:54	0.3 s	1/1	1/1

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Рисунок 3. Частина 3 - Completed Jobs (завершені завдання): 7, номери від 0 до 6.

Пояснення результату

Після групування та підрахунку додано cache() для збереження проміжного DataFrame. На скриншоті завдання 0 і 1 пов’язані з читанням CSV; завдання 2-5 з першим collect() у рядку 26 і заповненням кешу; завдання 6 з другим collect() у рядку 31. Загалом: 2 + 4 + 1 = 7.

cache() працює ліниво: саме позначення DataFrame для кешування не запускає весь розрахунок. Під час першого collect() результат обчислюється і кешується. Другий collect() застосовує фільтр count > 2 до збереженого результату, що дозволяє уникнути повторного виконання попередніх обчислень. unpersist() наприкінці звільняє ресурси кешу.

У порівнянні з частиною 2 кількість завдань зменшилася з 8 до 7. Кешування доцільне, коли той самий проміжний результат використовується декілька разів. Зазначені кількості завдань стосуються цих конкретних запусків і планів виконання Spark 3.5.5.